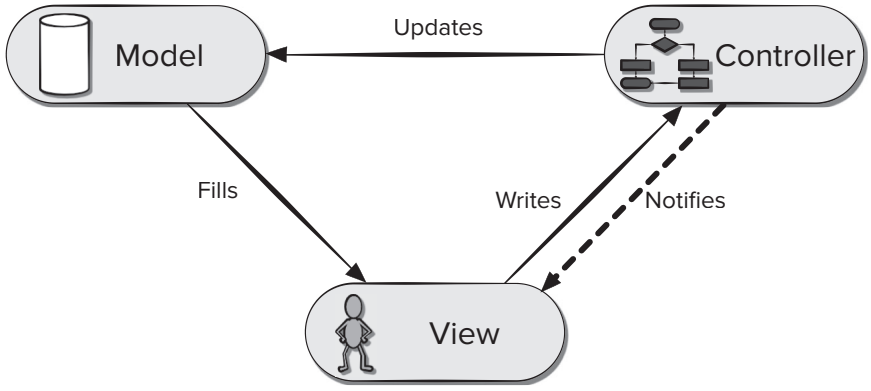
FIGURE 13.4
The MVC architecture



In many cases, WebApp architecture is defined within the context of the development environment in which the application is to be implemented. If you have further interest, see [Fow03] for a discussion of development environments and their role in the design of Web application architectures.

13.5.5 Navigation Design

Once the WebApp architecture has been established and the components (pages, scripts, applets, and other processing functions) of the architecture have been identified, you must define navigation pathways that enable users to access WebApp content and functions. To accomplish this, identify the semantics of navigation for different users of the site, and define the mechanics (syntax) of achieving the navigation.

Like many WebApp design actions, navigation design begins with a consideration of the user hierarchy and related use cases (Chapter 8) developed for each category of user (actor). Each actor may use the WebApp somewhat differently and therefore have different navigation requirements. In addition, the use cases developed for each actor will define a set of classes that encompass one or more content objects or WebApp functions. As each user interacts with the WebApp, she encounters a series of *navigation semantic units* (NSUs)—"a set of information and related navigation structures that collaborate in the fulfillment of a subset of related user requirements" [Cac02]. An NSU describes the navigation requirements for each use case. In essence, the NSU shows how an actor moves between content objects or WebApp functions.

An NSU is composed of a set of navigation elements called *ways of navigating* (WoN) [Gna99]. A WoN represents the best navigation pathway to achieve a navigational goal for a specific type of user. Each WoN is organized as a set of *navigational nodes* (NN) that are connected by navigational links. In some cases, a navigational link may be another NSU. Therefore, the overall navigation structure for a WebApp may be organized as a hierarchy of NSUs.

To illustrate the development of an NSU, consider the use case **Select SafeHome Components:**

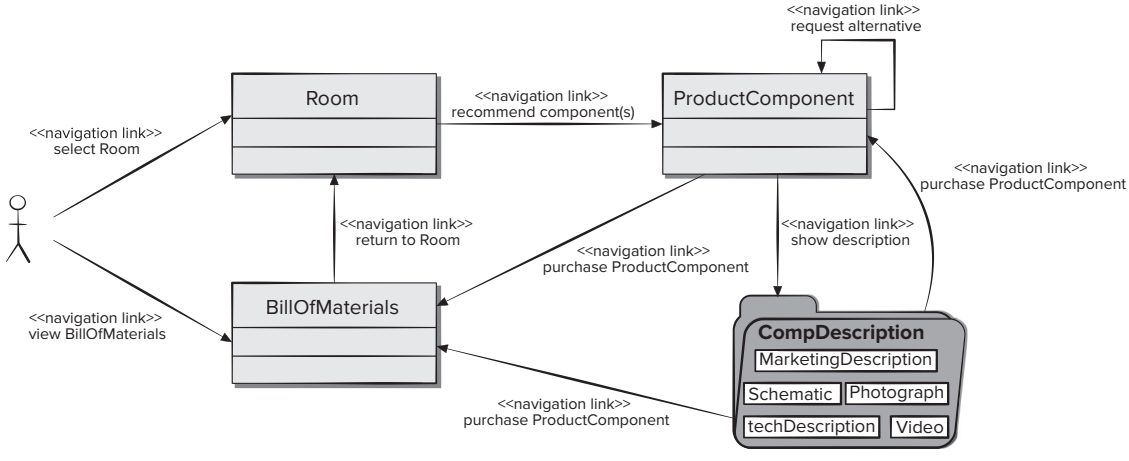
Use Case: *Select SafeHome Components*

The WebApp will recommend product components (e.g., control panels, sensors, cameras) and other features (e.g., PC-based functionality implemented in software) for each room and exterior entrance. If I request alternatives, the WebApp will provide them, if they exist. I will be able to get descriptive and pricing information for each product component. The WebApp will create and display a bill-of-materials as I select various components. I'll be able to give the bill-of-materials a name and save it for future reference (see use case **Save Configuration**).

The underlined items in the use case description represent classes and content objects that will be incorporated into one or more NSUs that will enable a new customer to perform the scenario described in the **Select SafeHome Components** use case.

Figure 13.5 depicts a partial semantic analysis of the navigation implied by the **Select SafeHome Components** use case. Using the terminology introduced earlier, the figure also represents a way of navigating (WoN) for the **SafeHomeAssured.com** WebApp. Important problem domain classes are shown, along with selected content objects (in this case the package of content objects named `CompDescription`, an attribute of the **ProductComponent** class). These items are navigation nodes. Each of the arrows represents a navigation link[24] and is labeled with the user-initiated action that causes the link to occur.

FIGURE 13.5 Creating an NSU



24 These are sometimes referred to as *navigation semantic links* (NSLs) [Cac02].

You can create an NSU for each use case associated with each user role. For example, a **new customer** for **SafeHomeAssured.com** may have three different use cases, all resulting in access to different information and WebApp functions. An NSU is created for each goal.

During the initial stages of navigation design, the WebApp content architecture is assessed to determine one or more WoN for each use case. As noted earlier, a WoN identifies navigation nodes (e.g., content) and then links that enable navigation between them. The WoN are then organized into NSUs.

As design proceeds, your next task is to define the mechanics of navigation. Most websites make use of one or more of the following navigation options for implementing each NSU: individual navigation links, horizontal or vertical navigation bars (lists), tabs, or access to a complete site map. If a site map is defined, it should be accessible from every page. The map itself should be organized so that the structure of WebApp information is readily apparent.

In addition to choosing the mechanics of navigation, you should also establish appropriate navigation conventions and aids. For example, icons and graphical links should look "clickable" by beveling the edges to give the image a three-dimensional look. Audio or visual feedback should be designed to provide the user with an indication that a navigation option has been chosen. For text-based navigation, color should be used to indicate navigation links and to provide an indication of links already traveled. These are but a few of dozens of design conventions that make navigation user-friendly.

13.6 COMPONENT-LEVEL DESIGN

Mobile apps deliver increasingly sophisticated processing functions that (1) perform localized processing to generate content and navigation capability in a dynamic fashion, (2) provide computation or data processing capability that are appropriate for the app's business domain, (3) provide sophisticated database query and access, and (4) establish data interfaces with external corporate systems. To achieve these (and many other) capabilities, you must design and construct program components that are identical in form to software components for traditional software.

The design methods discussed in Chapters 11 and 12 apply to mobile components with little, if any, modification. The implementation environment, programming languages, and design patterns, frameworks, and software may vary somewhat, but the overall design approach remains the same. To be cost conscious, you can design mobile components in such a way that they can be used without modification on several different mobile platforms.

13.7 MOBILITY AND DESIGN QUALITY

Every person has an opinion about what makes a "good" mobile app. Individual viewpoints vary widely. Some users enjoy flashy graphics; others want simple text. Some demand copious information; others desire an abbreviated presentation. Some like sophisticated analytical tools or database access; others like to keep it simple.

In fact, the user's perception of "goodness" (and the resultant acceptance or rejection of a mobile app as a consequence) might be more important than any technical discussion of mobile app quality. Mobile design quality attributes are virtually the same as WebApp quality characteristics.

But how is mobile quality perceived? What attributes must be exhibited to achieve goodness in the eyes of end users and at the same time exhibit the technical characteristics of quality that will enable you to correct, adapt, enhance, and support the mobile product over the long term?

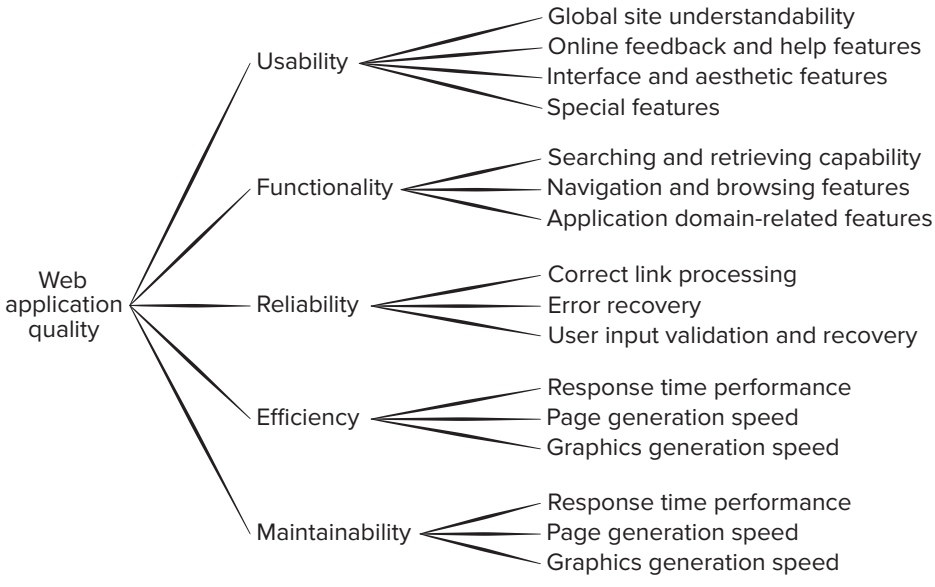
In reality, all the technical characteristics of design quality discussed in Chapter 12 and the generic quality attributes presented in Chapter 19 apply to mobile apps. However, the most relevant of these generic attributes—usability, functionality, reliability, efficiency, and maintainability—provide a useful basis for assessing the quality of mobile systems as well. Andreou [And05] suggests that end-user satisfaction with a mobile app is dictated by the same important quality factors—functionality, reliability, usability, efficiency, and maintainability—but adds portability to the list.

Olsina and his colleagues [Ols99] have prepared a "quality requirement tree" that identifies a set of technical attributes—usability, functionality, reliability, efficiency, and maintainability—that lead to high-quality mobile products.[25] Figure 13.6 summarizes their work. The criteria noted in the figure are of particular interest if you design, build, and maintain mobile products over the long term.

FIGURE 13.6

Quality requirements tree

Source: Olsina, Luis, Lafuente, Guillermo and Rossi, Gustavo, "Specifying Quality Characteristics and Attributes for Web Sites," Proceedings of the 1st International Conference on Software Engineering Workshop on Web Engineering, ACM, Los Angeles, May 1999.



25 These quality attributes are quite similar to those presented in Chapters 9 and 15. The implication is that quality characteristics are universal for all software.

Offutt [Off02] extends the five major quality attributes noted in Figure 13.6 by adding the following attributes:

Security. Mobile products have become heavily integrated with critical corporate and government databases. E-commerce applications extract and then store sensitive customer information. For these and many other reasons, mobile security is paramount in many situations. The key measure of security is the ability of the mobile app and its server environment to rebuff unauthorized access and/or thwart an outright malevolent attack. Security engineering is discussed in Chapter 18. For additional information on WebApp and mobile app security, see [Web13], [Pri10], [Vac06], and [Kiz05].

Availability. Even the best mobile product will not meet users' needs if it is unavailable. In a technical sense, availability is the measure of the percentage of time that a Web-based mobile resource is available for use. But Offutt [Off02] suggests that "using features available on only one browser or one platform" makes the mobile product unavailable to those with a different browser or platform configuration. The user will invariably go elsewhere.

Scalability. Can the mobile product and its server environment be scaled to handle 100, 1,000, 10,000, or 100,000 users? Will the app and the systems with which it is interfaced handle significant variation in volume, or will responsiveness drop dramatically (or cease altogether)? It is important to design a mobile environment that can accommodate the burden of success (i.e., significantly more end users) and become even more successful.

Time to Market. Although time to market is not a true quality attribute in the technical sense, it is a measure of quality from a business point of view. The first mobile product to address a specific market segment often captures a disproportionate number of end users.

Content Quality. Billions of Web pages are available for those in search of information. Even well-targeted Web searches result in an avalanche of content. With so many sources of information to choose from, how does the user assess the quality (e.g., veracity, accuracy, completeness, timeliness) of the content that is presented within a mobile product? This is part of what data science tries to address. The basics of data science are introduced in Appendix 2 of this book.

Tillman [Til00] suggests a useful set of criteria for assessing the quality of content: Can the scope and depth of content be easily determined to ensure that it meets the user's needs? Can the background and authority of the content's authors be easily identified? Is it possible to determine the currency of the content, the last update, and what was updated? Are the content and its location stable (i.e., will they remain at the referenced URL)? Is content credible? Is content unique? That is, does the mobile product provide some unique benefit to those who use it? Is content valuable to the targeted user community? Is content well organized? Indexed? Easily accessible? These questions represent only a small sampling of the issues that should be addressed as the design of a mobile product evolves.

INFO



Mobile Product—Quality Checklist

The following checklist provides a set of questions that will help both software engineers and end users assess overall mobile product quality:

- Can content, function, and/or navigation options be tailored to the user's preferences?
- Can content and/or functionality be customized to the bandwidth at which the user communicates? Does the app account for weak or lost signals in an acceptable manner?
- Can content, function, and/or navigation options be made context aware according to the user's preferences?
- Has adequate consideration been given to the power availability on the target device(s)?
- Have graphics, media (audio, video), and other Web or cloud services been used appropriately?
- Is the overall page design easy to read and navigate? Does the app take screen-size differences into account?
- Does the user interface conform to the display and interaction standards adopted for the targeted mobile device(s)?
- Does the app conform to the reliability, security, and privacy expectations of its users?
- What provisions have been made to ensure an app remains current?
- Has the mobile product been tested in all targeted user environments and for all targeted devices?

13.8 MOBILITY DESIGN BEST PRACTICES

There are several guidelines for developing mobile products[26] and for developing apps for specific platforms like Apple's iOS[27] or Google's Android.[28] Schumacher [Sch09] has collected many best practice ideas and has posted several specially adapted to the design of mobile applications and Web pages. Some important considerations when designing mobile touch-screen applications listed by Schumacher include:

- **Identify your audience.** The application must be written with the expectations and backgrounds of its users in mind. Experienced users want to do things quickly. Less experienced users will appreciate a handholding approach when they are first using the app.
- **Design for context of use.** It is important to consider how the user will interact with the real world while using the mobile product. Watching a movie on an airplane calls for a different user interface than checking the weather before you leave the office.
- **There is a fine line between simplicity and laziness.** Creating an intuitive user interface on a mobile device is much harder than simply removing features found in the user interface for the application running on a larger device. The user interface should provide all the information that enables a user to make her next decision.

26 See http://www.w3.org/TR/mwabp/.
27 See https://developer.apple.com/design/human-interface-guidelines/.
28 See http://developer.android.com/guide/components/index.html.